

Self-Parking Ackermann Vehicle

Marco Di Biasi, Francesco Granata

1. Introduction

This project presents a simulation of an Ackermann Vehicle, designed to perform an autonomous **three-step parking**. The vehicle setup follows an approximation of real-world physical parameters, aiming to keep the simulation as realistic as possible. The project is entirely developed in *Godot*, with its native programming language, *GDScript*, while more complex meshes are modelled in *Blender*.

The logic behind the physical model is based on the *Robotic Systems* course held by Professor Corrado Santoro and the related lecture slides.

The three phases of the parking maneuver are: reaching an intermediate target (referred to as `first_step_target` in the implementation), aligning the vehicle's rotation to match that of the parking slot, and entering the slot.

It is important to clarify that the maneuver is not designed to occur only in reverse; therefore, simulation runs where the vehicle performs a frontal parking may also occur.

2. System Initialization

2.1 Initial State Generation

The initial state of the system is **randomly generated**, including the planar position and y-axis rotation of both the vehicle and the parking slot. To avoid collisions during the initialization, a function called `get_non_overlapping_position(x, z, margin)` has been implemented. The margin is set large enough to ensure that each run of the simulation provides adequate spacing and avoids unwanted behavior when the vehicle is initialized too close to the `first_step_target`.

2.2 First Step Target

The variable `first_step_target` is a `Vector3` containing the coordinates of the intermediate target the vehicle is supposed to reach during the first phase of the parking maneuver. For the sake of visualization, the target is represented on the ground using a `SphereMesh`.

This imaginary point is calculated to lie on the line that passes through the center of the parking slot and shares its orientation, effectively placing it directly in front of it, as shown in Figure 1. The function `_calculate_firststep_target(pos, r_rad)` perform this computation, where the parameters represent the parking spot's planar position and its absolute orientation in radians. More precisely, the individual coordinates are computed as follows:

$$\begin{aligned} \text{target.x} &= \text{pos.x} + \sin(r) \cdot d \\ \text{target.z} &= \text{pos.z} + \cos(r) \cdot d \end{aligned}$$

Where d is an empirically chosen value representing the straight-line distance from the center of the parking slot to the target point.

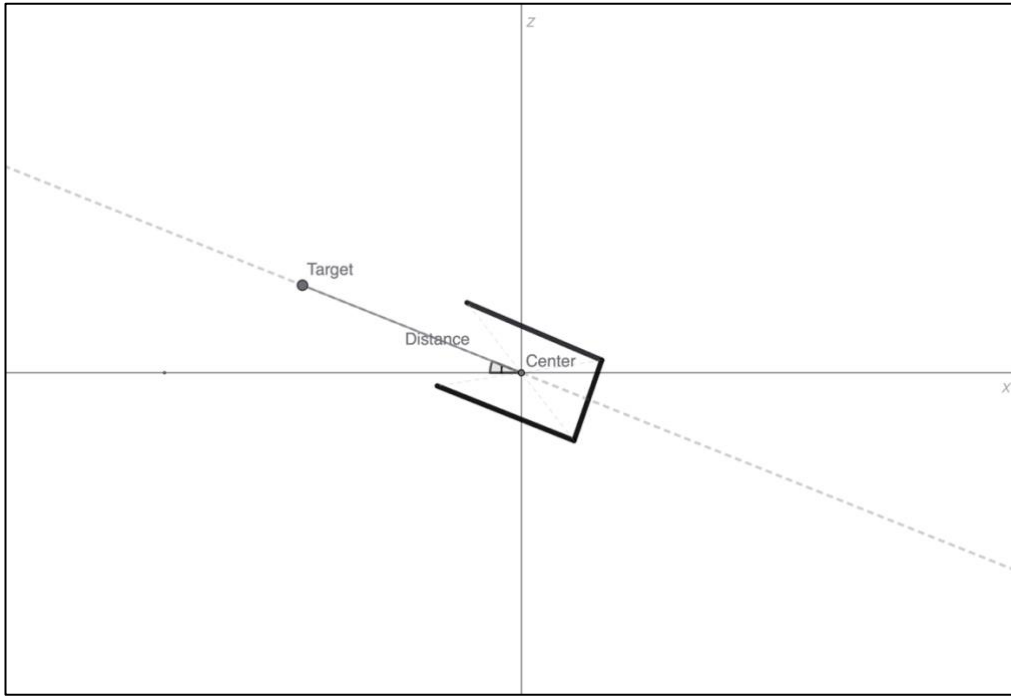


Figure 2.1: Target placement

2.3 Vehicle Components

The modules implemented in the project are based on the chain presented in Lecture 15, titled *Ackermann Steering*. The system structure is illustrated in Figure 2.

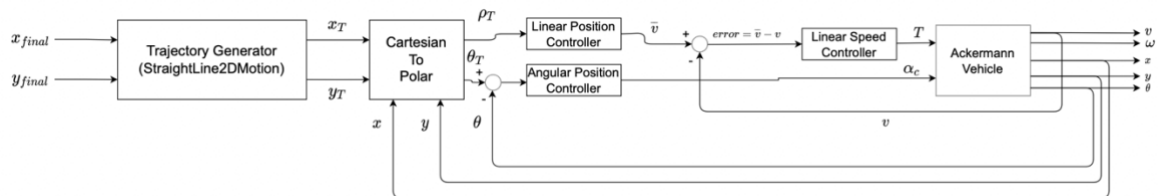


Figure 2: Ackermann Steering's chain of modules

The `_ready()` function in the `CarMain` class is a built-in callback invoked at the start of the simulation; it handles the initialization of all modules, using empirically chosen values for PID controllers and approximate values for the rest of the modules.

The chain implemented in this project include two **saturators**: one for limiting the maximum velocity during the `CRUISE` phase of the virtual robot (see Paragraph 3.1) and one for the vehicle's steering angle, which has a range of $[-30^\circ, 30^\circ]$ (or $[-\frac{\pi}{6}, \frac{\pi}{6}]$ in radians), as is typical for most real-world vehicles.

3. Modules Design and Implementation

3.1 Trajectory Generation

To reach the desired position, the vehicle uses a **virtual robot**, a trajectory generator that provide an ideal position to follow at each time step until the real target is reached. It is essential to determine a speed profile to be followed during motion, derived from setup variables and the **uniformly accelerated motion equations**:

$$\begin{aligned} \mathbf{a}(t) &= a \\ \mathbf{v}(t) &= \mathbf{v}(t_0) + a \cdot (t - t_0) \\ \mathbf{p}(t) &= \mathbf{p}(t_0) + \mathbf{v}(t_0) \cdot (t - t_0) + \frac{a}{2} \end{aligned}$$

And the **uniform motion equations**:

$$\begin{aligned} \mathbf{v}(t) &= \mathbf{v} \\ \mathbf{p}(t) &= \mathbf{p}(t_0) + \mathbf{v} \cdot (t - t_0) \end{aligned}$$

which are later discretized.

Additionally, the motion is divided into four distinct phases: acceleration, cruise, deceleration and target – referred to as ACCEL, CRUISE, DECEL and TARGET in the enumerator `motion_phase` inside the class `VirtualRobot`.

The acceleration phase consists in applying the uniformly accelerated motion equations for position and velocity, with acceleration treated as a constant parameter defined at setup. The transition from acceleration to cruise phase occurs when the maximum velocity is reached. During the cruise phase, the vehicle maintains this maximum velocity until the distance to the target becomes less than or equal to the deceleration distance ($p_{target} - p_{vehicle} \leq D$), which is calculated as follows:

$$D = \frac{v_{max}^2}{2 \cdot dec}$$

Where *dec* is a positive constant representing the deceleration, also defined at setup.

The deceleration phase operates similarly to the acceleration phase, using the deceleration constant instead. It continues until the vehicle reaches the final target, at which point it stops and transition to the target phase.

The vehicle operates in a two-dimensional space, whereas the virtual robot generates a one-dimensional trajectory. To integrate the two systems, the virtual robot's reference frame is roto-translated so that its trajectory aligns with the straight-line segment connecting the vehicle's starting position to the final target. At each time step, the resulting position, computed along this local frame, is then transformed back into the global $\{x, y\}$ reference frame and used as the target for the Polar Controller (see Paragraph 3.2). To implement this transformation, the class `StraightLine2DMotion` encapsulates the original `VirtualRobot` class.

3.2 Cartesian to Polar

The Cartesian-to-Polar module takes the desired intermediate position and the actual vehicle position and computes the error between them in polar coordinates. The $\{x, y\}$ coordinate system is converted to a $\{\rho, \theta\}$ system, where ρ represents the Euclidean distance between the points and θ is the angular deviation between the vehicle's orientation and the direction toward the target.

$$\rho = \sqrt{(x_{target} - x_{vehicle})^2 + (y_{target} - y_{vehicle})^2}$$

$$\theta = \arctan \frac{y_{target} - y_{vehicle}}{x_{target} - x_{vehicle}}$$

The calculated θ individuates the absolute heading angle towards the target. However, since the vehicle already has its own orientation, the correct heading error θ_{error} it is obtained by subtracting the vehicle's current heading from this angle.

$$\theta_{error} = \theta \ominus \theta_{vehicle}$$

The heading error is used to determine whether the vehicle has passed the target, since the distance ρ is always positive. Specifically, if the heading error falls within the second or third quadrant, it indicates that the target lies behind the vehicle. This logic is implemented by the `is_reverse` variable in the `CarMain` class, which signals when it is more convenient for the vehicle to move backwards. This class also includes the `_cartesian_2polar` function, which performs the coordinate transformation.

3.3 Controllers

The converted coordinates ρ and θ serve as inputs to the **linear position controller** and the **angular position controller**. They are both implemented using a PID controller—a module that takes an error and computes a control signal to reduce it. The linear position controller outputs a desired velocity, while the angular position controller outputs a desired orientation angle.

The PID consists of three different components: proportional, integral and derivative. Each contributes differently to the control action. The **proportional** term reacts to the current error and reduces it in proportion to its size, following the relation $P = K_p \cdot e(t)$, where K_p is a constant defined during setup. The **integral** term reacts to the accumulation of past errors, following the relation $I = K_i \cdot \int_0^t e(\tau) d\tau$, where K_i is a constant defined during setup. This term is particularly useful for eliminating small errors that persist over time. The **derivative** term reacts to the rate at which the error varies, following the relation $D = K_d \cdot \frac{de(t)}{dt}$, where K_d is a constant defined during setup. This term is particularly useful to predict the error's trend and adjust the control action accordingly. The three terms are then applied sequentially to compute the control signal sent to the vehicle.

$$\mathbf{u}(t) = P + I + D = K_p \cdot e(t) + K_i \cdot \int_0^t e(\tau) d\tau + K_d \cdot \frac{de(t)}{dt}$$

However, since the implementation works in discrete time steps, the integral and derivative terms cannot be directly computed and must be discretized. The integral becomes $I_k = K_i \sum_{i=0}^k e_i \Delta t$, and the derivative becomes $D_k = K_d \frac{e_k - e_{k-1}}{\Delta t}$. Putting all the components together, the control signal sent to the vehicle is:

$$\mathbf{u}_k = P_k + I_k + D_k = K_p \cdot e_k + K_i \sum_{i=0}^k e_i \Delta t + K_d \frac{e_k - e_{k-1}}{\Delta t}$$

For simplicity, the `LinearPID` and `AngularPID` classes are implemented separately in the code, although the structure is identical and only the type of error they process differs.

The output of the linear position controller is sequentially fed to the **linear speed controller** (as shown in Figure 2). This second PID controller has the same structure but takes as input the desired velocity– the difference between the desired velocity and the vehicle’s actual velocity. It then computes and returns the **torque T** , which is the rotational force applied to the wheels to adjust the vehicle speed.

3.4 Ackermann Steering

The Ackermann steering model, commonly adopted in most cars, consists of a single traction motor combined with a mechanism to steers the front wheels. A *differential* is required to allow the wheels to rotate at different speeds during turns. The geometric model shown in Figure 3 is useful to derive the equations required to control the vehicle.

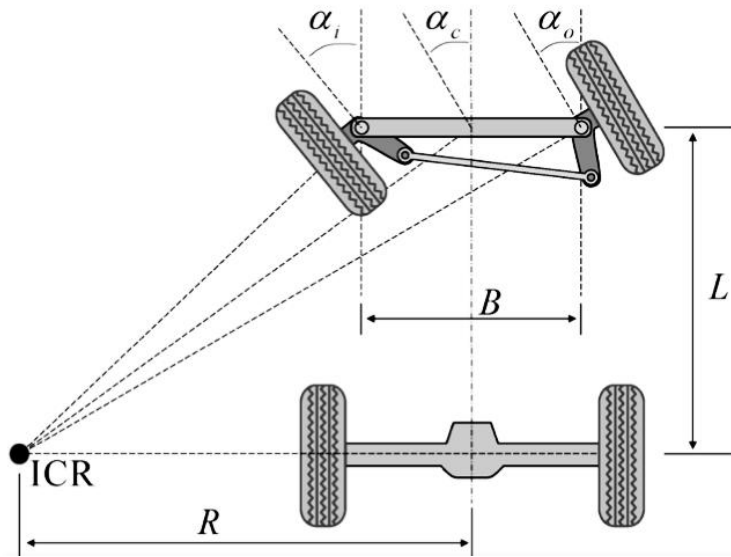


Figure 3: Ackermann steering geometric model

The parameters shown in Figure 3 are:

- **ICR**: Instantaneous Centre of Rotation
- **R**: Radius of rotation
- **B**: Wheelbase
- **L**: Lateral wheelbase
- α_i : Inner wheel steering angle
- α_o : Outer wheel steering angle
- α_c : Centre (virtual) wheel steering angle

The vehicle moves with a linear speed v and an angular speed $\omega = \frac{v}{R}$, which depends on v . The radius of rotation is given by $R = \frac{L}{\tan(\alpha_c)}$. The input to the model is not directly a speed, but rather a torque T , which is converted into a traction force by $F = \frac{T}{r_{wheel}}$. Based on this, the dynamics is modelled as follows:

$$v' = -\frac{b}{M}v + \frac{F}{M}$$

where b represents the viscous friction coefficient and M represents the mass of the vehicle.

All the constants defining the physical model of the vehicle are specified when the `AckermannVehicle` class is initialized. Since ω depends on α_c , it is more natural for an Ackermann vehicle to receive the desired steering angle as input rather than of the desired angular speed.

4. Physics Process

4.1 Implementation of the Module Chain